

DIGITAL VERIFICATION DIPLOMA

Assignment 6 extra

NAME:	Roaa Atef Mohamed
-------	-------------------

1. ALSU DESIGN:

```
1  import shared_pkg::*;
2
3  module ALSU(alsu_if.DUT a_if);
4  reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
5  reg [1:0] cin_reg;
6  reg [2:0] opcode_reg;
7  reg signed [2:0] A_reg, B_reg;
8  reg signed [5:0] shift_out_reg;
9  wire invalid_red_op, invalid_opcode, invalid;
10 //Invalid handling
11 assign invalid_red_op = (red_op_A_reg || red_op_B_reg) & (opcode_reg[1] || opcode_reg[2]);
12 assign invalid_opcode = opcode_reg[1] & opcode_reg[2];
13 assign invalid = invalid_red_op || invalid_opcode;
14 //Registering input signals
15 always @(posedge a_if.clk or posedge a_if.rst) begin
16     if(a_if.rst) begin
17         cin_reg <= 0;
18         red_op_B_reg <= 0;
19         red_op_A_reg <= 0;
20         bypass_B_reg <= 0;
21         bypass_A_reg <= 0;
22         direction_reg <= 0;
23         serial_in_reg <= 0;
24         opcode_reg <= 0;
25         A_reg <= 0;
26         B_reg <= 0;
27     end else begin
28         cin_reg <= a_if.cin;
29         red_op_B_reg <= a_if.red_op_B;
30         red_op_A_reg <= a_if.red_op_A;
31         bypass_B_reg <= a_if.bypass_B;
32         bypass_A_reg <= a_if.bypass_A;
33         direction_reg <= a_if.direction;
34         serial_in_reg <= a_if.serial_in;
35         opcode_reg <= a_if.opcode;
36         A_reg <= a_if.A;
37         B_reg <= a_if.B;
38     end
39 end
40
41 //leds output blinking
42 always @(posedge a_if.clk or posedge a_if.rst) begin
43     if(a_if.rst) begin
44         a_if.leds <= 0;
45     end else begin
46         if (invalid)
47             a_if.leds <= ~a_if.leds;
48         else
49             a_if.leds <= 0;
50     end
51 end
52
53 //ALSU output processing
54 always @(posedge a_if.clk or posedge a_if.rst) begin
55     if(a_if.rst) begin
```

```

56     a_if.out <= 0;
57 end
58 else begin
59     if (bypass_A_reg && bypass_B_reg)
60         a_if.out <= (INPUT_PRIORITY == "A")? A_reg: B_reg;
61     else if (bypass_A_reg)
62         a_if.out <= A_reg;
63     else if (bypass_B_reg)
64         a_if.out <= B_reg;
65     else if (invalid)
66         a_if.out <= 0;
67     else begin
68         case (opcode_reg)
69             3'h0: begin
70                 if (red_op_A_reg && red_op_B_reg)
71                     a_if.out <= (INPUT_PRIORITY == "A")? |A_reg: |B_reg;
72                 else if (red_op_A_reg)
73                     a_if.out <= |A_reg;
74                 else if (red_op_B_reg)
75                     a_if.out <= |B_reg;
76                 else
77                     a_if.out <= A_reg | B_reg;
78             end
79             3'h1: begin
80                 if (red_op_A_reg && red_op_B_reg)
81                     a_if.out <= (INPUT_PRIORITY == "A")? ^A_reg: ^B_reg;
82                 else if (red_op_A_reg)
83                     a_if.out <= ^A_reg;
84                 else if (red_op_B_reg)
85                     a_if.out <= ^B_reg;
86                 else
87                     a_if.out <= A_reg ^ B_reg;
88             end
89             3'h2: begin
90                 if (FULL_ADDER == "ON")
91                     a_if.out <= A_reg + B_reg + cin_reg;
92             else
93                 a_if.out <= A_reg + B_reg;
94             end
95             3'h3: a_if.out <= A_reg * B_reg;
96             3'h4: begin
97
98                 a_if.out <= shift_out_reg;
99             end
100             3'h5: begin
101
102                 a_if.out <= shift_out_reg;
103             end
104         endcase
105     end
106 end
107 end
108
109 endmodule
110

```

2. alsu agent:

```

1  package alsu_agent_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4      import alsu_sequencer_pkg::*;
5      import alsu_monitor_pkg::*;
6      import alsu_seq_item_pkg::*;
7      import alsu_driver_pkg::*;
8      import alsu_config_pkg::*;
9
10     class alsu_agent extends uvm_agent;
11         `uvm_component_utils(alsu_agent)
12         alsu_sequencer sqr;
13         alsu_driver driver;
14         alsu_monitor monitor;
15         alsu_config alsu_cfg;
16         uvm_analysis_port #(alsu_seq_item) agent_ap;
17
18         function new (string name = "shift_reg_agent", uvm_component parent = null);
19             super.new(name,parent);
20         endfunction
21
22         function void build_phase(uvm_phase phase);
23             super.build_phase(phase);
24             alsu_cfg = alsu_config::type_id::create("alsu_cfg");
25             agent_ap = new("agent_ap", this);
26             if (!uvm_config_db #(alsu_config)::get(this, "", "CFG", alsu_cfg))begin
27                 `uvm_fatal("build_phase", "Unable to get configuration object")
28             end
29             if(alsu_cfg.is_active==UVM_ACTIVE) begin
30                 sqr = alsu_sequencer::type_id::create("sqr",this);
31                 driver = alsu_driver::type_id::create("driver",this);
32                 monitor = alsu_monitor::type_id::create("monitor",this);
33             end
34             else begin
35                 monitor = alsu_monitor::type_id::create("monitor",this);
36             end
37         endfunction
38
39         function void connect_phase(uvm_phase phase);
40             if(alsu_cfg.is_active==UVM_ACTIVE) begin
41                 driver.alsu_vif = alsu_cfg.alsu_vif;
42                 monitor.alsu_vif = alsu_cfg.alsu_vif;
43                 driver.seq_item_port.connect(sqr.seq_item_export);
44                 monitor.monitor_ap.connect(agent_ap);
45             end
46             else begin
47                 monitor.alsu_vif = alsu_cfg.alsu_vif;
48                 monitor.monitor_ap.connect(agent_ap);
49             end
50         endfunction
51     endclass
52
53 endpackage
54
55

```

3. alsu config:

```
1 package alsu_config_pkg;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4
5   class alsu_config extends uvm_object;
6     `uvm_object_utils(alsu_config)
7
8     virtual alsu_if alsu_vif;
9     uvm_active_passive_enum is_active;
10    function new(string name = "alsu_config");
11      super.new(name);
12    endfunction
13  endclass
14 endpackage
```

4. alsu coverage :

```

1 package alsu_coverage_pkg;
2 import uvm_pkg::*;
3 import alsu_seq_item_pkg::*;
4 import shared_pkg::*;
5 `include "uvm_macros.svh"
6 class alsu_coverage extends uvm_component;
7     `uvm_component_utils(alsu_coverage)
8     uvm_analysis_export #(alsu_seq_item) coverage_export;
9     uvm_tlm_analysis_fifo #(alsu_seq_item) cov_fifo;
10    alsu_seq_item seq_item_cov;
11    /*.....cover group.....*/
12    covergroup alsu_cg ;
13    /*.....coverpoint for a maxpos,maxneg,0 values.....*/
14    A_cp1: coverpoint seq_item_cov.A
15    {
16        bins A_data_0 = {0};
17        bins A_data_max = {MAXPOS};
18        bins A_data_min = {MAXNEG};
19        bins A_data_default=default;
20    }
21    /*.....coverpoint for when takes walkingone.....*/
22    A_cp2: coverpoint seq_item_cov.A iff(seq_item_cov.red_op_a)
23    {
24        bins walkingones []={3'b001,3'b010,3'b100};
25    }
26    /*.....coverpoint for b maxpos,maxneg,0 values.....*/
27    B_cp1: coverpoint seq_item_cov.B
28    {
29        bins B_data_0 = {0};
30        bins B_data_max = {MAXPOS};
31        bins B_data_min = {MAXNEG};
32        bins B_data_default=default;
33    }
34    /*.....coverpoint for when takes walkingone.....*/
35    B_cp2: coverpoint seq_item_cov.B iff(!seq_item_cov.red_op_a&seq_item_cov.red_op_b)
36    {
37        bins walkingones []={3'b001,3'b010,3'b100};
38    }
39    //////////////////////////////////***opcode coverpoint**////////////////////////////////
40    ALU_cp:coverpoint seq_item_cov.op_code
41    {
42        bins Bins_shift[] = {SHIFT,ROTATE};
43        bins Bins_arith [] = {ADD,MULT};
44        bins Bins_bitwise [] = {OR,XOR};
45        bins Bins_invalid ={INVALID_6,INVALID_7};
46        bins Bins_trans = (OR=>XOR=>ADD=>MULT=>SHIFT=>ROTATE);
47    }
48    c_cp:coverpoint seq_item_cov.cin
49    {
50        bins one = {1};
51        bins zero = {0};
52    }
53
54    direction_cp:coverpoint seq_item_cov.direction
55    {

```

with width 4

```

56     bins one = {1};
57     bins zero = {0};
58 }
59     serial_cp:coverpoint seq_item_cov.serial_in
60 {
61     bins one = {1};
62     bins zero = {0};
63 }
64     reda_cp:coverpoint seq_item_cov.red_op_a
65 {
66     bins one = { 1};
67     bins zero = {0};
68 }
69     redb_cp:coverpoint seq_item_cov.red_op_b
70 {
71     bins one = {1};
72     bins zero = {0};
73 }
74 c1_cs : cross B_cp1,ALU_cp
75 {
76     bins add_mult_b1 = binsof(ALU_cp.Bins_arith) && binsof(B_cp1.B_data_max);
77     bins add_mult_b2= binsof(ALU_cp.Bins_arith)&& binsof(B_cp1.B_data_0);
78     bins add_mult_b3 = binsof(ALU_cp.Bins_arith)&&binsof(B_cp1.B_data_min);
79     option.cross_auto_bin_max=0;
80 }
81 c2_cs : cross A_cp1,ALU_cp
82 {
83     bins add_mult_a1 = binsof(ALU_cp.Bins_arith) && binsof(A_cp1.A_data_max);
84     bins add_mult_a2= binsof(ALU_cp.Bins_arith)&& binsof(A_cp1.A_data_0);
85     bins add_mult_a3 = binsof(ALU_cp.Bins_arith)&&binsof(A_cp1.A_data_min);
86     option.cross_auto_bin_max=0;
87 }
88 c3_cs : cross ALU_cp,c_cp
89 {
90     bins add = binsof(ALU_cp) intersect {ADD} && (binsof(c_cp.zero)||binsof(c_cp.one));
91     option.cross_auto_bin_max=0;
92 }
93 c4_cs : cross ALU_cp,direction_cp
94 {
95     bins sh = binsof (ALU_cp) intersect {SHIFT} && (binsof (direction_cp.zero)||binsof (direction_cp.one));
96     bins r = binsof (ALU_cp) intersect {ROTATE} && (binsof (direction_cp.zero)||binsof (direction_cp.one));
97     option.cross_auto_bin_max=0;
98 }
99 c5_cs : cross ALU_cp,serial_cp
100 {
101     bins sh = binsof (ALU_cp) intersect {SHIFT} && (binsof (serial_cp.zero)||binsof (serial_cp.one));
102     option.cross_auto_bin_max=0;
103 }
104 c6_cs : cross ALU_cp,reda_cp
105 {
106     bins reda = binsof (ALU_cp.Bins_bitwise) && (binsof (reda_cp.one));
107     option.cross_auto_bin_max=0;
108 }
109 c7_cs : cross ALU_cp,redb_cp
110 {

```



```

99  c5_cs : cross ALU_cp,serial_cp
100 {
101     bins sh = binsof (ALU_cp) intersect {SHIFT} && (binsof (serial_cp.zero)||binsof (serial_cp.one));
102     option.cross_auto_bin_max=0;
103 }
104  c6_cs : cross ALU_cp,reda_cp
105 {
106     bins reda = binsof (ALU_cp.Bins_bitwise) && (binsof (reda_cp.one));
107     option.cross_auto_bin_max=0;
108 }
109  c7_cs : cross ALU_cp,redb_cp
110 {
111     bins redb = binsof (ALU_cp.Bins_bitwise) && (binsof (redb_cp.one));
112     option.cross_auto_bin_max=0;
113 }
114  c8_cs : cross ALU_cp,redb_cp,reda_cp
115 {
116     bins a1 = binsof(ALU_cp.Bins_shift) &&(binsof(reda_cp.one))&&(binsof(redb_cp.zero));
117     bins a2 = binsof(ALU_cp.Bins_arith) &&(binsof(reda_cp.one))&&(binsof(redb_cp.zero));
118     bins b1 = binsof(ALU_cp.Bins_shift) &&(binsof(redb_cp.one))&&(binsof(reda_cp.zero));
119     bins b2 = binsof(ALU_cp.Bins_arith) &&(binsof(redb_cp.one))&&(binsof(reda_cp.zero));
120     bins ab1 = binsof(ALU_cp.Bins_shift) &&(binsof(redb_cp.one))&&(binsof(reda_cp.one));
121     bins ab2 = binsof(ALU_cp.Bins_arith) &&(binsof(redb_cp.one))&&(binsof(reda_cp.one));
122     option.cross_auto_bin_max=0;
123 }
124 endgroup
125 /*.....*/
126 function new(string name = "alsu_coverage", uvm_component parent = null) ;
127     super.new(name, parent);
128     alsu_cg = new();
129 endfunction
130
131 function void build_phase(uvm_phase phase);
132     super.build_phase(phase);
133     coverage_export = new("coverage_export", this);
134     cov_fifo = new("cov_fifo", this);
135 endfunction
136
137 function void connect_phase(uvm_phase phase);
138     super.connect_phase(phase);
139     coverage_export.connect(cov_fifo.analysis_export);
140 endfunction
141
142 task run_phase(uvm_phase phase);
143     super.run_phase(phase);
144     forever begin
145         cov_fifo.get(seq_item_cov);
146         if (!seq_item_cov.rst||seq_item_cov.bypass_a||seq_item_cov.bypass_b)) begin
147             alsu_cg.sample();
148         end
149     end
150 endtask
151 endclass
152 endpackage

```

5. Als driver:


```

1 package alsu_driver_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5
6     import alsu_seq_item_pkg::*;
7     import alsu_config_pkg::*;
8
9     class alsu_driver extends uvm_driver #(alsu_seq_item);
10         `uvm_component_utils(alsu_driver)
11
12         virtual alsu_if alsu_vif;
13         alsu_seq_item stim_seq_item;
14
15         function new(string name = "alsu_driver", uvm_component parent = null);
16             super.new(name, parent);
17         endfunction
18
19         task run_phase(uvm_phase phase);
20             super.run_phase(phase);
21             forever begin
22                 stim_seq_item = alsu_seq_item::type_id::create("stim_seq_item");
23                 seq_item_port.get_next_item(stim_seq_item);
24                 alsu_vif.rst=stim_seq_item.rst;
25                 alsu_vif.serial_in=stim_seq_item.serial_in;
26                 alsu_vif.direction=stim_seq_item.direction;
27                 alsu_vif.cin=stim_seq_item.cin;
28                 alsu_vif.opcode= stim_seq_item.op_code;
29                 alsu_vif.red_op_A= stim_seq_item.red_op_a;
30                 alsu_vif.red_op_B=stim_seq_item.red_op_b;
31                 alsu_vif.bypass_A= stim_seq_item.bypass_a;
32                 alsu_vif.bypass_B= stim_seq_item.bypass_b;
33                 alsu_vif.A= stim_seq_item.A;
34                 alsu_vif.B= stim_seq_item.B;
35                 repeat (2) @(negedge alsu_vif.clk);
36                 seq_item_port.item_done();
37                 `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
38             end
39         endtask
40
41     endclass
42 endpackage

```

6. Alsu env:

```

1 package alsu_env_pkg;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import alsu_agent_pkg::*;
5   import alsu_scoreboard_pkg::*;
6   import alsu_coverage_pkg::*;
7   class alsu_env extends uvm_env;
8     `uvm_component_utils(alsu_env)
9
10    alsu_agent agent;
11    alsu_scoreboard score_b;
12    alsu_coverage coverage;
13
14    function new(string name = "shift_reg_env", uvm_component parent = null);
15      super.new(name,parent);
16    endfunction
17
18    function void build_phase(uvm_phase phase);
19      super.build_phase(phase);
20      agent = alsu_agent::type_id::create("agent",this);
21      score_b = alsu_scoreboard::type_id::create("score_b",this);
22      coverage = alsu_coverage::type_id::create("coverage",this);
23    endfunction: build_phase
24
25    function void connect_phase(uvm_phase phase);
26      agent.agent_ap.connect(score_b.sb_export);
27      agent.agent_ap.connect(coverage.coverage_export);
28    endfunction
29  endclass
30 endpackage
31
32
33

```

7. alsu if :

```

1 import shared_pkg::*;
2
3 interface alsu_if (clk);
4   input clk;
5   logic cin, rst, red_op_A, red_op_B, bypass_A, bypass_B, direction, serial_in;
6   OP_CODE opcode;
7   logic signed [2:0] A, B;
8   logic [15:0] leds;
9   logic signed [5:0] out;
10  logic signed [5:0] expected_out;
11  logic [15:0] leds_expected;
12  modport DUT (input clk,cin,rst,red_op_A,red_op_B,bypass_A,bypass_B,direction,serial_in,opcode,A,B,output leds,out);
13  modport GOLDEN (
14    input clk, cin, rst, red_op_A, red_op_B, bypass_A, bypass_B,
15    direction, serial_in, opcode, A, B,
16    output expected_out, leds_expected
17  );
18 endinterface
19

```

8. Alsu monitor:

```

1 package alsu_monitor_pkg;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import shared_pkg::*;
5   import alsu_seq_item_pkg::*;
6   class alsu_monitor extends uvm_monitor;
7     `uvm_component_utils(alsu_monitor)
8     virtual alsu_if alsu_vif;
9     alsu_seq_item rsp_seq_item;
10    uvm_analysis_port #(alsu_seq_item) monitor_ap;
11
12    function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
13      super.new(name, parent);
14    endfunction
15    function void build_phase(uvm_phase phase);
16      super.build_phase(phase);
17      monitor_ap = new("monitor_ap", this);
18    endfunction: build_phase
19    task run_phase(uvm_phase phase);
20      super.run_phase(phase);
21      forever begin
22        rsp_seq_item = alsu_seq_item::type_id::create("rsp_seq_item");
23        repeat(2) @(negedge alsu_vif.clk);
24        rsp_seq_item.rst=alsu_vif.rst;
25        rsp_seq_item.serial_in=alsu_vif.serial_in;
26        rsp_seq_item.direction=alsu_vif.direction;
27        rsp_seq_item.cin=alsu_vif.cin;
28        rsp_seq_item.op_code=alsu_vif.opcode;
29        rsp_seq_item.red_op_a=alsu_vif.red_op_A;
30        rsp_seq_item.red_op_b=alsu_vif.red_op_B;
31        rsp_seq_item.bypass_a=alsu_vif.bypass_A;
32        rsp_seq_item.bypass_b=alsu_vif.bypass_B;
33        rsp_seq_item.A=alsu_vif.A;
34        rsp_seq_item.B=alsu_vif.B;
35        rsp_seq_item.out=alsu_vif.out;
36        rsp_seq_item.leds=alsu_vif.leds;
37        rsp_seq_item.expected_out=alsu_vif.expected_out;
38        rsp_seq_item.leds_expected=alsu_vif.leds_expected;
39        monitor_ap.write(rsp_seq_item);
40        `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
41      end
42    endtask
43  endclass
44 endpackage

```

9. Alsu main seq :

```

1 package alsu_main_seq_pkg;
2   import uvm_pkg::*;
3   `include "uvm_macros.svh"
4   import alsu_seq_item_pkg::*;
5   import shared_pkg::*;
6   class alsu_main_sequence extends uvm_sequence #(alsu_seq_item);
7     `uvm_object_utils(alsu_main_sequence);
8     alsu_seq_item seq_item;
9     function new(string name = "alsu_main_sequence");
10       super.new(name);
11     endfunction
12     task body;
13       repeat(20000) begin
14         seq_item = alsu_seq_item::type_id::create("seq_item");
15         //disable constraints number 0/////
16         seq_item.X.constraint_mode(0);
17         start_item(seq_item);
18         assert(seq_item.randomize());
19         finish_item(seq_item);
20       end
21       repeat(1000) begin
22         seq_item = alsu_seq_item::type_id::create("seq_item");
23         //disable all constraints /////
24         seq_item.constraint_mode(0);
25         //enable constraint X /////
26         seq_item.X.constraint_mode(1);
27         assert(seq_item.randomize());
28         //directed value for some inputs/////
29         seq_item.rst=0;
30         seq_item.bypass_a=0;
31         seq_item.bypass_b=0;
32         seq_item.red_op_a=0;
33         seq_item.red_op_b=0;
34         for (int i=0;i<10;i++) begin
35           start_item(seq_item);
36           seq_item.op_code=seq_item.array[i];
37           finish_item(seq_item);
38         end
39       end
40     endtask
41  endclass
42 endpackage

```

10. Alsu reset seq:

```

1 package alsu_reset_seq_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5 import shared_pkg::*;
6 import alsu_seq_item_pkg::*;
7 class alsu_reset_sequence extends uvm_sequence #(alsu_seq_item);
8     `uvm_object_utils(alsu_reset_sequence);
9     alsu_seq_item seq_item;
10
11     function new (string name = "alsu_reset_sequence");
12         super.new(name);
13     endfunction
14
15     task body;
16         seq_item = alsu_seq_item::type_id::create("seq_item");
17         start_item(seq_item);
18         seq_item.rst = 1;
19         seq_item.serial_in=0;
20         seq_item.A=0;
21         seq_item.B=0;
22         seq_item.cin=0;
23         seq_item.direction=0;
24         seq_item.op_code=OR;
25         seq_item.red_op_a=0;
26         seq_item.red_op_b=0;
27         seq_item.bypass_a=0;
28         seq_item.bypass_b=0;
29         finish_item(seq_item);
30     endtask
31 endclass
32 endpackage

```

11. Alsu scoreboard:

```

1 package alsu_scoreboard_pkg;
2 import uvm_pkg::*;
3 include "uvm_macros.svh"
4 import alsu_seq_item_pkg::*;
5 import shared_pkg::*;
6 /*.....expected out declaration.....*/
7 Logic signed [5:0] expected_out;
8 Logic [15:0] leds_expected;
9 class alsu_scoreboard extends uvm_scoreboard;
10     uvm_component_utils(alsu_scoreboard)
11     uvm_analysis_export #(alsu_seq_item) sb_export;
12     uvm_tlm_analysis_fifo #(alsu_seq_item) sb_fifo;
13     alsu_seq_item seq_item_sb;
14     int error_count = 0;
15     int correct_count = 0;
16     /*.....*/
17     function new(string name = "alsu_scoreboard", uvm_component parent = null);
18         super.new(name, parent);
19     endfunction
20
21     function void build_phase(uvm_phase phase);
22         super.build_phase(phase);
23         sb_export = new("sb_export", this);
24         sb_fifo = new("sb_fifo", this);
25     endfunction
26
27     function void connect_phase(uvm_phase phase);
28         super.connect_phase(phase);
29         sb_export.connect(sb_fifo.analysis_export);
30     endfunction
31
32     task run_phase(uvm_phase phase);
33         super.run_phase(phase);
34         forever begin
35             sb_fifo.get(seq_item_sb);
36             if ((seq_item_sb.out == seq_item_sb.expected_out) && (seq_item_sb.leds == seq_item_sb.leds_expected)) begin
37                 uvm_info("run_phase", $sformatf("%s", seq_item_sb.convert2string()), UVM_LOW);
38                 uvm_info("run_phase", $sformatf("expected_out value : %b, leds_expected : %b", seq_item_sb.expected_out, seq_item_sb.leds_expected), UVM_
39                 correct_count++;
40             end
41             else begin
42                 uvm_error("run_phase", $sformatf("Comparsion failed, Transaction received by the DUT:%s While the expected out:%0b%0b & leds_
43                 error_count++;
44             end
45         end
46     endtask
47
48     function void report_phase(uvm_phase phase);
49         super.report_phase(phase);
50         uvm_info("report_phase", $sformatf("Total successful transactions: %0d", correct_count), UVM_LOW);
51         uvm_info("report_phase", $sformatf("Total failed transactions: %0d", error_count), UVM_LOW);
52     endfunction
53 endclass
54 endpackage

```

12. Alsu seq item :

```

1 package alsu_seq_item_pkg;
2
3 import uvm_pkg::*;
4 import shared_pkg::*;
5 include "uvm_macros.svh"
6
7 /*.....class.....*/
8 class alsu_seq_item extends uvm_sequence_item;
9     uvm_object_utils(alsu_seq_item)
10
11     /*.....define variables.....*/
12     rand Logic rst;
13     rand Logic signed [2:0] A,B;
14     rand OP_CODE op_code;
15     rand Logic cin,serial_in,direction;
16     rand Logic red_op_a,red_op_b;
17     rand Logic bypass_a,bypass_b;
18     rand OP_CODE array [6];
19     Logic signed [5:0] out;
20     Logic [15:0] leds;
21     Logic signed [5:0] expected_out;
22     Logic [15:0] leds_expected;
23
24     /*.....*/
25
26     function new(string name = "alsu_seq_item");
27         super.new(name);
28     endfunction
29
30     function string convert2string();
31         return $sformatf("%s rst = %b, A = %d, B = %d,op_code= %s, cin = %b,serial_in=%b,direction=%b,red_op_a=%b,red_op_b=%b,bypass_a=
32     endfunction
33
34     function string convert2string_stimulus();
35         return $sformatf(" rst = %b, A = %d, B = %d,op_code= %s, cin = %b,serial_in=%b,direction=%b,red_op_a=%b,red_op_b=%b,bypass_a=
36     endfunction
37
38     /*.....constraints.....*/
39     /*.....constraint to force rst to be low most of the time..*/
40     constraint RST {
41         rst dist{0:-99,1:=1};
42     }
43     /*.....op code constraint.....*/
44     constraint OP {
45         op_code dist{OR:=30,XOR:=10,[ADD:ROTATE]:/45,[INVALID_6:INVALID_7]:/5};
46     }
47     /*.....constraint A,B.....*/
48     constraint A_B {
49         /*.....forc A,B to be MAX,MIN,ZERO 90% of the time.....*/
50         if (op_code==ADD||op_code==MULT)
51         {
52             A dist{ MAXNEG:=20,MAXPOS:=40,0:=30, [-3:-1]:/5,[1:2]:/5};
53             B dist{ MAXNEG:=20,MAXPOS:=40,0:=30, [-3:-1]:/5,[1:2]:/5};
54         }
55     }
56     /*.....constraints on A when OR,XOR opcode and red_op_a=1.....*/

```



```

53 }
54 /*.....constraints on A when OR,XOR opcode and red_op_a=1.....*/
55 if((op_code==OR||op_code==XOR)&red_op_a)
56 {
57   A dist {3'b001:=20,3'b010:=25,3'b100:=40,0:=5,3:=5,[-4:-1]:/5};
58   B==0;
59 }
60 /*.....constraints on B when OR,XOR opcode and red_op_b=1.....*/
61 if((op_code==OR||op_code==XOR)&red_op_b)
62 {
63   B dist { 3'b001:=20,3'b010:=25,3'b100:=40, 0:=5,3:=5,[-4:-1]:/5};
64   A==0;
65 }
66 }
67
68 /*.....red_op_A,red_op_B constraints.....*/
69 constraint red_op_A {
70   red_op_a dist {0:=5,1:=95};
71 }
72 constraint red_op_B {
73   red_op_b dist {0:=5,1:=95};
74 }
75 /*.....bypass A,bypass B must be disabled most of the time.....*/
76 constraint bypass_A {
77   bypass_a dist {0:=95,1:=5};
78 }
79 constraint bypass_B {
80   bypass_b dist {0:=95,1:=5};
81 }
82 //////////////////////////////////////////////////unique constraint for the array////////////////////////////////////
83 constraint X {
84   foreach (array[i]){
85     array[i]!={INVALID_6,INVALID_7};
86     if(i==0)
87       array[i]==OR;
88     else if (i > 0) {
89       (array[i-1] == OR) -> (array[i] == XOR);
90       (array[i-1] == XOR) -> (array[i] == ADD);
91       (array[i-1] == ADD) -> (array[i] == MULT);
92       (array[i-1] == MULT) -> (array[i] == SHIFT);
93       (array[i-1] == SHIFT) -> (array[i] == ROTATE);
94     }
95   }
96 }
97 }
98
99 endclass
100
101
102 endpackage

```

13. Alsu seqr:

```

1 package alsu_sequencer_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import alsu_seq_item_pkg::*;
7 class alsu_sequencer extends uvm_sequencer #(alsu_seq_item);
8   `uvm_component_utils(alsu_sequencer);
9
10   function new (string name = "alsu_sequencer", uvm_component parent = null);
11     super.new(name, parent);
12   endfunction
13 endclass
14
15 endpackage

```

14. Alsu sva :


```

1  import shared_pkg::*;
2  module alsu_sva (alsu_if_out a_if);
3  *.....OUT assertion.....*/
4  a1 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A & a_if.bypass_B & INPUT_PRIORITY == "A") |> ##2 (a_if.out == $past(a_if.A,2)));
5  c1 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A & a_if.bypass_B
6  INPUT_PRIORITY == "A") |> ##2 (a_if.out == $past(a_if.A,2)) );
7
8  a2 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A & !a_if.bypass_B
9  ) |> ##2 (a_if.out == $past(a_if.A,2)) );
10 c2 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (a_if.bypass_A
11 & !a_if.bypass_B) |> ##2 (a_if.out == $past(a_if.A,2)) );
12
13 a3 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & a_if.bypass_B
14 ) |> ##2 (a_if.out == $past(a_if.B,2)) );
15 c3 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & a_if.bypass_B
16 ) |> ##2 (a_if.out == $past(a_if.B,2)) );
17
18 a4 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) ((a_if.bypass_A & !a_if.bypass_B
19 & ((a_if.red_op_A & a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
20 |> ##2 (a_if.out == 0) ));
21 c4 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) ((a_if.bypass_A & !a_if.bypass_B
22 & ((a_if.red_op_A & a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
23 |> ##2 (a_if.out == 0) ));
24
25 a5 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
26 & !a_if.bypass_B & a_if.opcode == OR & a_if.red_op_A & a_if.red_op_B & INPUT_PRIORITY == "A") |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
27 c5 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
28 & !a_if.bypass_B & a_if.opcode == OR & a_if.red_op_A & a_if.red_op_B & INPUT_PRIORITY == "A") |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
29
30 a6 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
31 & !a_if.bypass_B & a_if.opcode == OR & !a_if.red_op_A & a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2)) & (a_if.leds==0) );
32 c6 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A
33 & !a_if.bypass_B & a_if.opcode == OR & !a_if.red_op_A & a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2)) & (a_if.leds==0) );
34
35 a7 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
36 & a_if.opcode == OR & a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
37 c7 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
38 & a_if.opcode == OR & a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
39
40 a8 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
41 & a_if.opcode == OR & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.A,2) | $past(a_if.B,2)) & (a_if.leds==0) );
42 c8 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
43 & a_if.opcode == OR & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2) | $past(a_if.A,2)) & (a_if.leds==0) );
44
45 a9 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
46 & a_if.opcode == XOR & a_if.red_op_A & a_if.red_op_B & INPUT_PRIORITY == "A") |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
47 c9 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
48 & a_if.opcode == XOR & a_if.red_op_A & a_if.red_op_B & INPUT_PRIORITY == "A") |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
49
50 a10 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
51 & a_if.opcode == XOR & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2)) & (a_if.leds==0) );
52 c10 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
53 & a_if.opcode == XOR & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2)) & (a_if.leds==0) );
54
55 a11 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
56 & a_if.opcode == XOR & a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
57 c11 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
58 & a_if.opcode == XOR & a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.A,2)) & (a_if.leds==0) );
59
60 a12 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
61 & a_if.opcode == XOR & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2) | $past(a_if.A,2)) & (a_if.leds==0) );
62 c12 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
63 & a_if.opcode == XOR & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2) | $past(a_if.A,2)) & (a_if.leds==0) );
64
65 a13 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
66 & a_if.opcode == ADD & !a_if.red_op_A & !a_if.red_op_B & FULL_ADDER=="0") |> ##2 (a_if.out == $past(a_if.B,2) + $past(a_if.cin,2)) & (a_if.leds==0) );
67 c13 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
68 & a_if.opcode == ADD & !a_if.red_op_A & !a_if.red_op_B & FULL_ADDER=="0") |> ##2 (a_if.out == $past(a_if.B,2) + $past(a_if.cin,2)) & (a_if.leds==0) );
69
70 a14 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
71 & a_if.opcode == MULT & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2) * $past(a_if.A,2)) & (a_if.leds==0) );
72 c14 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
73 & a_if.opcode == MULT & !a_if.red_op_A & !a_if.red_op_B) |> ##2 (a_if.out == $past(a_if.B,2) * $past(a_if.A,2)) & (a_if.leds==0) );
74
75 a15 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
76 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==SHIFT & a_if.direction==1) |> ##2 (a_if.out==($past(a_if.out[4:0],1), $past(a_if.serial_in,2))) & (a_if.leds==0) );
77 c15 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
78 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==SHIFT & a_if.direction==1) |> ##2 (a_if.out==($past(a_if.out[4:0],1), $past(a_if.serial_in,2))) & (a_if.leds==0) );
79
80 a16 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
81 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==SHIFT & a_if.direction==0) |> ##2 (a_if.out==($past(a_if.serial_in,2), $past(a_if.out[5:1],1))) & (a_if.leds==0) );
82 c16 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
83 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==SHIFT & a_if.direction==0) |> ##2 (a_if.out==($past(a_if.serial_in,2), $past(a_if.out[5:1],1))) & (a_if.leds==0) );
84
85 a17 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
86 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==ROTATE & a_if.direction==1) |> ##2 (a_if.out == $past(a_if.out[4:0],1), $past(a_if.out[5:1],1))) & (a_if.leds==0) );
87 c17 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
88 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==ROTATE & a_if.direction==1) |> ##2 (a_if.out == $past(a_if.out[4:0],1), $past(a_if.out[5:1],1))) & (a_if.leds==0) );
89
90 a18 : assert property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B & !a_if.red_op_A
91 & !a_if.red_op_B & a_if.opcode==ROTATE & a_if.direction==0) |> ##2 (a_if.out == $past(a_if.out[0],1), $past(a_if.out[5:1],1))) & (a_if.leds==0) );
92 c18 : cover property (@(posedge a_if.clk) disable iff(a_if.rst) (!a_if.bypass_A & !a_if.bypass_B
93 & !a_if.red_op_A & !a_if.red_op_B & a_if.opcode==ROTATE & a_if.direction==0) |> ##2 (a_if.out == $past(a_if.out[0],1), $past(a_if.out[5:1],1))) & (a_if.leds==0) );
94
95 19: assert property (@(posedge a_if.clk) disable iff(a_if.rst) ( ((a_if.red_op_A & !a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
96 |> ##2 (a_if.leds == $past(a_if.leds,1)) );
97
98 19: cover property (@(posedge a_if.clk) disable iff(a_if.rst) ( ((a_if.red_op_A & !a_if.red_op_B) & (a_if.opcode != OR & a_if.opcode != XOR))
99 |> ##2 (a_if.leds == $past(a_if.leds,1)) );
100
101 *.....reset assertion.....*/
102 *.....reset assertion.....*/
103 always_comb begin
104 if(a_if.rst)
105 1:assert final (a_if.out==0 & a_if.leds==0);
106 end
107 endmodule

```

15. Golden model:

```

1 import shared_pkg::*;
2 module golden_design (alsu_if.GOLDEN a_if);
3     reg red_op_A_reg_ex, red_op_B_reg_ex, bypass_A_reg_ex, bypass_B_reg_ex, direction_reg_ex, serial_in_reg_ex;
4     reg [1:0] cin_reg_ex;
5     reg [2:0] opcode_reg_ex;
6     reg signed [2:0] A_reg_ex, B_reg_ex;
7     wire invalid_red_op_ex, invalid_opcode_ex, invalid_ex;
8     //Invalid handling
9     assign invalid_red_op_ex = (red_op_A_reg_ex || red_op_B_reg_ex) & (opcode_reg_ex[1] || opcode_reg_ex[2]);
10    assign invalid_opcode_ex = opcode_reg_ex[1] & opcode_reg_ex[2];
11    assign invalid = invalid_red_op_ex || invalid_opcode_ex;
12    //Registering input signals
13    always @(posedge a_if.clk or posedge a_if.rst) begin
14        if(a_if.rst) begin
15            cin_reg_ex <= 0;
16            red_op_B_reg_ex <= 0;
17            red_op_A_reg_ex <= 0;
18            bypass_B_reg_ex <= 0;
19            bypass_A_reg_ex <= 0;
20            direction_reg_ex <= 0;
21            serial_in_reg_ex <= 0;
22            opcode_reg_ex <= 0;
23            A_reg_ex <= 0;
24            B_reg_ex <= 0;
25        end else begin
26            cin_reg_ex <= a_if.cin;
27            red_op_B_reg_ex <= a_if.red_op_B;
28            red_op_A_reg_ex <= a_if.red_op_A;
29            bypass_B_reg_ex <= a_if.bypass_B;
30            bypass_A_reg_ex <= a_if.bypass_A;
31            direction_reg_ex <= a_if.direction;
32            serial_in_reg_ex <= a_if.serial_in;
33            opcode_reg_ex <= a_if.opcode;
34            A_reg_ex <= a_if.A;
35            B_reg_ex <= a_if.B;
36        end
37    end
38
39    //leds output blinking
40    always @(posedge a_if.clk or posedge a_if.rst) begin
41        if(a_if.rst) begin
42            a_if.leds_expected <= 0;
43        end else begin
44            if (invalid)
45                a_if.leds_expected <= ~a_if.leds_expected;
46            else
47                a_if.leds_expected <= 0;
48        end
49    end
50
51    //ALSU output processing
52    always @(posedge a_if.clk or posedge a_if.rst) begin
53        if(a_if.rst) begin
54            a_if.expected_out <= 0;
55        end
56    end
57
58    else begin
59        if (bypass_A_reg_ex && bypass_B_reg_ex)
60            a_if.expected_out <= (INPUT_PRIORITY == "A")? A_reg_ex: B_reg_ex;
61        else if (bypass_A_reg_ex)
62            a_if.expected_out <= A_reg_ex;
63        else if (bypass_B_reg_ex)
64            a_if.expected_out <= B_reg_ex;
65        else if (invalid)
66            a_if.expected_out <= 0;
67        else begin
68            case (opcode_reg_ex)
69                3'h0: begin
70                    if (red_op_A_reg_ex && red_op_B_reg_ex)
71                        a_if.expected_out <= (INPUT_PRIORITY == "A")? |A_reg_ex: |B_reg_ex;
72                    else if (red_op_A_reg_ex)
73                        a_if.expected_out <= |A_reg_ex;
74                    else if (red_op_B_reg_ex)
75                        a_if.expected_out <= |B_reg_ex;
76                    else
77                        a_if.expected_out <= A_reg_ex | B_reg_ex;
78                end
79                3'h1: begin
80                    if (red_op_A_reg_ex && red_op_B_reg_ex)
81                        a_if.expected_out <= (INPUT_PRIORITY == "A")? ^A_reg_ex: ^B_reg_ex;
82                    else if (red_op_A_reg_ex)
83                        a_if.expected_out <= ^A_reg_ex;
84                    else if (red_op_B_reg_ex)
85                        a_if.expected_out <= ^B_reg_ex;
86                    else
87                        a_if.expected_out <= A_reg_ex ^ B_reg_ex;
88                end
89                3'h2: begin
90                    if(FULL_ADDER=="ON")
91                        a_if.expected_out <= A_reg_ex + B_reg_ex;
92                    else
93                        a_if.expected_out <= A_reg_ex + B_reg_ex;
94                end
95                3'h3: a_if.expected_out <= A_reg_ex * B_reg_ex;
96                3'h4: begin
97                    if (direction_reg_ex)
98                        a_if.expected_out <= {a_if.expected_out[4:0], serial_in_reg_ex};
99                    else
100                        a_if.expected_out <= {serial_in_reg_ex, a_if.expected_out[5:1]};
101                end
102                3'h5: begin
103                    if (direction_reg_ex)
104                        a_if.expected_out <= {a_if.expected_out[4:0], a_if.expected_out[5]};
105                    else
106                        a_if.expected_out <= {a_if.expected_out[0], a_if.expected_out[5:1]};
107                end
108            endcase
109        end
110    end
111 endmodule

```

16. test:

```

1 package alsu_test_pkg;
2 import uvm_pkg::*;
3 `include "uvm_macros.svh"
4 import alsu_config_pkg::*;
5 import shift_reg_env_pkg::*;
6 import shift_reg_config_pkg::*;
7 import alsu_main_seq_pkg::*;
8 import alsu_reset_seq_pkg::*;
9 import alsu_agent_pkg::*;
10 import shift_reg_main_seq_pkg::*;
11 import shift_reg_agent_pkg::*;
12 import alsu_env_pkg::*;
13 class alsu_test extends uvm_test;
14     `uvm_component_utils(alsu_test)
15     alsu_agent agent;
16     alsu_config alsu_cfg;
17     shift_reg_config shift_cfg;
18     virtual alsu_if alsu_vif;
19     virtual shift_reg_if shift_reg_vif;
20     alsu_main_sequence main_seq;
21     alsu_reset_sequence reset_seq;
22     alsu_env a_env;
23     shift_reg_env s_env;
24
25     function new(string name = "alsu_test", uvm_component parent = null);
26         super.new(name, parent);
27     endfunction
28
29     function void build_phase(uvm_phase phase);
30         super.build_phase(phase);
31         agent = alsu_agent::type_id::create("agent", this);
32         a_env = alsu_env::type_id::create("a_env", this);
33         s_env = shift_reg_env::type_id::create("s_env", this);
34         alsu_cfg = alsu_config::type_id::create("alsu_cfg");
35         shift_cfg = shift_reg_config::type_id::create("shift_cfg");
36         main_seq = alsu_main_sequence::type_id::create("main_seq", this);
37         reset_seq = alsu_reset_sequence::type_id::create("reset_seq", this);
38         /*.....alsu db.....*/
39         if (!uvm_config_db #(virtual alsu_if)::get(this, "", "ALSU_IF", alsu_cfg.alsu_vif))
40             `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the alsu from the uvm_config_db");
41
42         /*.....shift reg db.....*/
43         if (!uvm_config_db #(virtual shift_reg_if)::get(this, "", "SHIFT_IF", shift_cfg.shift_reg_vif))
44             `uvm_fatal("build_phase", "Test - Unable to get the virtual interface of the shift from the uvm_config_db");
45         uvm_config_db #(shift_reg_config)::set(this, "", "CFG", shift_cfg);
46         uvm_config_db #(alsu_config)::set(this, "", "CFG", alsu_cfg);
47         /*.....*/
48         shift_cfg.is_active = UVM_PASSIVE;
49         alsu_cfg.is_active = UVM_ACTIVE;
50     endfunction
51     task run_phase(uvm_phase phase);
52         super.run_phase(phase);
53         phase.raise_objection(this);
54         //reset sequence
55         `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
56
57         alsu_cfg.is_active = UVM_ACTIVE;
58     endfunction
59     task run_phase(uvm_phase phase);
60         super.run_phase(phase);
61         phase.raise_objection(this);
62         //reset sequence
63         `uvm_info("run_phase", "Reset Asserted", UVM_LOW)
64         reset_seq.start(agent.sqr);
65         `uvm_info("run_phase", "Reset Deasserted", UVM_LOW)
66         //main sequence
67         `uvm_info("run_phase", "Stimulus Generation Started", UVM_LOW)
68         main_seq.start(agent.sqr);
69         `uvm_info("run_phase", "Stimulus Generation Ended", UVM_LOW)
70         phase.drop_objection(this);
71     endtask
72 endclass
73 endpackage

```

Tab Si

17. top:


```

alsu_test_pkg.sv x alsu_top.sv x shift_reg_if.sv x alsu_if.sv x alsu_monitor_pkg.sv x alsu_reset_seq
1  import uvm_pkg::*;
2  `include "uvm_macros.svh"
3  import alsu_test_pkg::*;
4  module alsu_top();
5      // Clock generation
6      bit clk;
7      initial begin
8          forever #1 clk=~clk;
9      end
10     // Instantiate the interface and DUT
11     alsu_if a_if (clk);
12     ALSU dut ( a_if);
13     golden_design ex ( a_if);
14     bind ALSU alsu_sva sva_inst (a_if);
15     shift_reg_if shiftregif();
16     shift_reg DUT(shiftregif);
17     assign shiftregif.datain=a_if.out;
18     assign dut.shift_out_reg=shiftregif.dataout;
19     assign shiftregif.serial_in=dut.serial_in_reg;
20     assign shiftregif.direction=dut.direction_reg;
21     assign shiftregif.mode=dut.opcode_reg[0];
22     initial begin
23         //.....configuration database.....//
24         uvm_config_db #(virtual alsu_if) ::set (null,"uvm_test_top","ALSU_IF",a_if);
25         uvm_config_db #(virtual shift_reg_if)::set(null, "uvm_test_top", "SHIFT_IF", shiftregif);
26         //.....uvm run task.....//
27         run_test("alsu_test");
28     end
29 endmodule
30
31

```

18. Shared pkg:

```

shift_reg_drive shift_reg_monitor_pkg.sv x shift_reg_sequencer_pkg.sv x shift_reg_agent_pkg.sv
1  package shared_pkg;
2      /*.....parameters.....*/
3      parameter INPUT_PRIORITY="A";
4      parameter FULL_ADDER="ON";
5      parameter MAXPOS=3;
6      parameter MAXNEG=-4;
7      typedef enum {OR,XOR,ADD,MULT,SHIFT,ROTATE,INVALID_6,INVALID_7} OP_CODE;
8      typedef enum {RIGHT, LEFT} direction_e;
9  endpackage
10

```

19. Shift reg design:

```

5 // User specified shift register design
6 //
7 ///////////////////////////////////////////////////////////////////
8 module shift_reg (shift_reg_if.DUT shiftregif);
9
10
11 always @(*) begin
12
13     if (shiftregif.mode) // rotate
14         if (shiftregif.direction) // left
15             shiftregif.dataout = {shiftregif.datain[4:0], shiftregif.datain[5]};
16         else
17             shiftregif.dataout = {shiftregif.datain[0], shiftregif.datain[5:1]};
18     else // shift
19         if (shiftregif.direction) // left
20             shiftregif.dataout = {shiftregif.datain[4:0], shiftregif.serial_in};
21         else
22             shiftregif.dataout = {shiftregif.serial_in, shiftregif.datain[5:1]};
23     end
24 endmodule

```

20. Shft reg agent :

```

1  package shift_reg_agent_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4
5      import shift_reg_sequencer_pkg::*;
6      import shift_reg_monitor_pkg::*;
7      import shift_reg_seq_item_pkg::*;
8      import shift_reg_driver_pkg::*;
9      import shift_reg_config_pkg::*;
10
11     class shift_reg_agent extends uvm_agent;
12         `uvm_component_utils(shift_reg_agent)
13         shift_reg_sequencer sqr_s;
14         shift_reg_driver drv;
15         shift_reg_monitor mon;
16         shift_reg_config shift_reg_cfg;
17         uvm_analysis_port #(shift_reg_seq_item) agt_ap;
18
19         function new (string name = "shift_reg_agent", uvm_component parent = null);
20             super.new(name,parent);
21         endfunction
22
23         function void build_phase(uvm_phase phase);
24             super.build_phase(phase);
25             shift_reg_cfg = shift_reg_config::type_id::create("shift_reg_cfg");
26             agt_ap = new("agt_ap", this);
27             if (!uvm_config_db #(shift_reg_config)::get(this, "", "CFG", shift_reg_cfg))begin
28                 `uvm_fatal("build_phase", "Unable to get configuration object")
29             end
30             if(shift_reg_cfg.is_active==UVM_ACTIVE) begin
31                 sqr_s = shift_reg_sequencer::type_id::create("sqr_s",this);
32                 drv = shift_reg_driver::type_id::create("drv",this);
33                 mon = shift_reg_monitor::type_id::create("mon",this);
34             end
35             else begin
36                 mon = shift_reg_monitor::type_id::create("mon",this);
37             end
38         endfunction
39
40         function void connect_phase(uvm_phase phase);
41             if(shift_reg_cfg.is_active==UVM_ACTIVE) begin
42                 drv.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
43                 drv.seq_item_port.connect(sqr_s.seq_item_export);
44                 mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
45                 mon.mon_ap.connect(agt_ap);
46             end
47         else begin
48             mon.shift_reg_vif = shift_reg_cfg.shift_reg_vif;
49             mon.mon_ap.connect(agt_ap);
50         end
51         endfunction
52     endclass
53
54 endpackage

```

21. Shift reg config:

```

1  package shift_reg_config_pkg;
2      import uvm_pkg::*;
3      `include "uvm_macros.svh"
4
5      class shift_reg_config extends uvm_object;
6          `uvm_object_utils(shift_reg_config)
7
8          virtual shift_reg_if shift_reg_vif;
9          uvm_active_passive_enum is_active;
10         function new(string name = "shift_reg_config");
11             super.new(name);
12         endfunction
13     endclass
14 endpackage : shift_reg_config_pkg

```


22. Shift reg coverage :

```
1 package shift_reg_coverage_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import shift_reg_seq_item_pkg::*;
7
8 class shift_reg_coverage extends uvm_component;
9     `uvm_component_utils(shift_reg_coverage)
10     uvm_analysis_export #(shift_reg_seq_item) cov_export;
11     uvm_tlm_analysis_fifo #(shift_reg_seq_item) cov_fifo;
12     shift_reg_seq_item seq_item_cov;
13
14     covergroup cvr_grp;
15         datain_cvp : coverpoint seq_item_cov.datain;
16         serialin_cvp : coverpoint seq_item_cov.serial_in;
17         direction_cvp : coverpoint seq_item_cov.direction;
18         mode_cvp : coverpoint seq_item_cov.mode;
19     endgroup
20
21     function new(string name = "shift_reg_coverage", uvm_component parent = null) ;
22         super.new(name, parent);
23         cvr_grp = new();
24     endfunction
25
26     function void build_phase(uvm_phase phase);
27         super.build_phase(phase);
28         cov_export = new("cov_export", this);
29         cov_fifo = new("cov_fifo", this);
30     endfunction
31
32     function void connect_phase(uvm_phase phase);
33         super.connect_phase(phase);
34         cov_export.connect(cov_fifo.analysis_export);
35     endfunction
36
37     task run_phase(uvm_phase phase);
38         super.run_phase(phase);
39         forever begin
40             cov_fifo.get(seq_item_cov);
41             cvr_grp.sample();
42         end
43     endtask
44
45 endclass
46 endpackage
```

23. Shift reg driver:

```

1 package shift_reg_driver_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import shift_reg_seq_item_pkg::*;
7 import shift_reg_config_pkg::*;
8
9 class shift_reg_driver extends uvm_driver #(shift_reg_seq_item);
10   `uvm_component_utils(shift_reg_driver)
11
12   virtual shift_reg_if shift_reg_vif;
13   shift_reg_seq_item stim_seq_item;
14
15   function new(string name = "shift_reg_driver", uvm_component parent = null);
16     super.new(name, parent);
17   endfunction
18
19   task run_phase(uvm_phase phase);
20     super.run_phase(phase);
21     forever begin
22       stim_seq_item = shift_reg_seq_item::type_id::create("stim_seq_item");
23       seq_item_port.get_next_item(stim_seq_item);
24       shift_reg_vif.direction = stim_seq_item.direction;
25       shift_reg_vif.mode = stim_seq_item.mode;
26       shift_reg_vif.datain = stim_seq_item.datain;
27       shift_reg_vif.serial_in = stim_seq_item.serial_in;
28       #2;
29       seq_item_port.item_done();
30       `uvm_info("run_phase", stim_seq_item.convert2string_stimulus(), UVM_HIGH)
31     end
32   endtask
33
34   endclass
35 endpackage

```

24. Shift reg env:

```

1 package shift_reg_env_pkg;
2
3     import uvm_pkg::*;
4     `include "uvm_macros.svh"
5
6     import shift_reg_agent_pkg::*;
7     import shift_reg_scoreboard_pkg::*;
8     import shift_reg_coverage_pkg::*;
9     class shift_reg_env extends uvm_env;
10         `uvm_component_utils(shift_reg_env)
11
12         shift_reg_agent agt;
13         shift_reg_scoreboard sb;
14         shift_reg_coverage cov;
15
16         function new(string name = "shift_reg_env", uvm_component parent = null);
17             super.new(name,parent);
18         endfunction
19
20         function void build_phase(uvm_phase phase);
21             super.build_phase(phase);
22             agt = shift_reg_agent::type_id::create("agt",this);
23             sb = shift_reg_scoreboard::type_id::create("sb",this);
24             cov = shift_reg_coverage::type_id::create("cov",this);
25         endfunction: build_phase
26
27         function void connect_phase(uvm_phase phase);
28             agt.agt_ap.connect(sb.sb_export);
29             agt.agt_ap.connect(cov.cov_export);
30         endfunction
31     endclass
32 endpackage
33
34
35

```

25. Shift reg if :

```

1 interface shift_reg_if ();
2
3     bit serial_in, direction, mode;
4     logic [5:0] datain;
5     logic [5:0] dataout;
6
7     modport DUT (input serial_in, direction, mode, datain, output dataout);
8
9
10 endinterface
11
12

```

26. Shift reg main seq:

```

1  package shift_reg_main_seq_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      import shift_reg_seq_item_pkg::*;
7      import shared_pkg::*;
8      class shift_reg_main_sequence extends uvm_sequence #(shift_reg_seq_item);
9          `uvm_object_utils(shift_reg_main_sequence);
10         shift_reg_seq_item seq_item;
11
12         function new (string name = "shift_reg_main_sequence");
13             super.new(name);
14         endfunction
15
16         task body;
17             repeat(26000) begin
18                 seq_item = shift_reg_seq_item::type_id::create("seq_item");
19                 start_item(seq_item);
20                 assert(seq_item.randomize());
21                 finish_item(seq_item);
22             end
23         endtask
24     endclass
25 endpackage

```

27. Shift reg monitor :

```

1  package shift_reg_monitor_pkg;
2
3      import uvm_pkg::*;
4      `include "uvm_macros.svh"
5
6      import shared_pkg::*;
7
8      import shift_reg_seq_item_pkg::*;
9      class shift_reg_monitor extends uvm_monitor;
10         `uvm_component_utils(shift_reg_monitor)
11         virtual shift_reg_if shift_reg_vif;
12         shift_reg_seq_item rsp_seq_item;
13         uvm_analysis_port #(shift_reg_seq_item) mon_ap;
14
15         function new(string name = "shift_reg_monitor", uvm_component parent = null) ;
16             super.new(name, parent);
17         endfunction
18
19         function void build_phase(uvm_phase phase);
20             super.build_phase(phase);
21             mon_ap = new("mon_ap", this);
22
23         endfunction: build_phase
24
25         task run_phase(uvm_phase phase);
26             super.run_phase(phase);
27             forever begin
28                 rsp_seq_item = shift_reg_seq_item::type_id::create("rsp_seq_item");
29                 #2;
30                 rsp_seq_item.direction = direction_e'(shift_reg_vif.direction);
31                 rsp_seq_item.mode = (shift_reg_vif.mode);
32                 rsp_seq_item.datain = shift_reg_vif.datain;
33                 rsp_seq_item.serial_in = shift_reg_vif.serial_in;
34                 rsp_seq_item.dataout = shift_reg_vif.dataout;
35                 mon_ap.write(rsp_seq_item);
36                 `uvm_info("run_phase", rsp_seq_item.convert2string(), UVM_HIGH)
37             end
38         endtask
39     endclass
40 endpackage : shift_reg_monitor_pkg

```

28. Shift reg scoreboard:


```

1 package shift_reg_scoreboard_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import shift_reg_seq_item_pkg::*;
7 import shared_pkg::*;
8
9 class shift_reg_scoreboard extends uvm_scoreboard;
10     uvm_component_utils(shift_reg_scoreboard);
11     uvm_analysis_export #(shift_reg_seq_item) sb_export;
12     uvm_tlm_analysis_fifo #(shift_reg_seq_item) sb_fifo;
13     shift_reg_seq_item seq_item_sb;
14     logic [5:0] shift_reg_out_ref;
15
16     int error_count = 0;
17     int correct_count = 0;
18
19     function new(string name = "shift_reg_scoreboard", uvm_component parent = null);
20         super.new(name, parent);
21     endfunction
22
23     function void build_phase(uvm_phase phase);
24         super.build_phase(phase);
25         sb_export = new("sb_export", this);
26         sb_fifo = new("sb_fifo", this);
27     endfunction
28
29     function void connect_phase(uvm_phase phase);
30         super.connect_phase(phase);
31         sb_export.connect(sb_fifo.analysis_export);
32     endfunction
33
34     task run_phase(uvm_phase phase);
35         super.run_phase(phase);
36         forever begin
37             sb_fifo.get(seq_item_sb);
38             ref_model(seq_item_sb);
39             if (seq_item_sb.dataout != shift_reg_out_ref) begin
40                 `uvm_error("run_phase", $sformatf("Comparison failed, Transaction received by the DUT:%s While the reference out:0b%0b",
41                     seq_item_sb.dataout, shift_reg_out_ref));
42                 error_count++;
43             end
44             else begin
45                 `uvm_info("run_phase", $sformatf("Correct shift_reg out: %s ", seq_item_sb.convert2string()), UVM_HIGH);
46                 correct_count++;
47             end
48         end
49     endtask
50
51     task ref_model(shift_reg_seq_item seq_item_chk);
52         case (seq_item_chk.mode)
53             SHIFT:
54                 if (seq_item_chk.direction == LEFT)
55                     shift_reg_out_ref = {seq_item_chk.datain[4:0], seq_item_chk.serial_in};
56                 else if (seq_item_chk.direction == RIGHT)
57                     shift_reg_out_ref = {seq_item_chk.serial_in, seq_item_chk.datain[5:1]};
58             ROTATE:
59                 if (seq_item_chk.direction == LEFT)
60                     shift_reg_out_ref = {seq_item_chk.datain[4:0], seq_item_chk.datain[5]};
61                 else if (seq_item_chk.direction == RIGHT)
62                     shift_reg_out_ref = {seq_item_chk.datain[0], seq_item_chk.datain[5:1]};
63         endcase
64     endtask
65
66     function void report_phase(uvm_phase phase);
67         super.report_phase(phase);
68         `uvm_info("report_phase", $sformatf("Total successful transactions: %0d", correct_count), UVM_LOW);
69         `uvm_info("report_phase", $sformatf("Total failed transactions: %0d", error_count), UVM_LOW);
70     endfunction
71
72 endclass
73 endpackage : shift_reg_scoreboard_pkg

```

29. Shift reg seq item:

```
1 package shift_reg_seq_item_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import shared_pkg::*;
7
8 class shift_reg_seq_item extends uvm_sequence_item;
9     `uvm_object_utils(shift_reg_seq_item)
10
11     rand bit serial_in;
12     rand direction_e direction;
13     rand bit mode;
14     rand bit [5:0] datain;
15     logic [5:0] dataout;
16
17     function new(string name = "shift_reg_seq_item");
18         super.new(name);
19     endfunction
20
21     function string convert2string();
22         return $formatf("%s serial_in = %0xb, direction = %s, mode = %s, datain = %0xb", super.convert2string(), serial_in, direction.name(), mode, datain);
23     endfunction
24
25     function string convert2string_stimulus();
26         return $formatf(" serial_in = %0xb, direction = %s, mode = %s, datain = %0xb", serial_in, direction.name(), mode, datain);
27     endfunction
28 endclass
29
30
31 endpackage
```

30. Shift reg sqr:

```
1 package shift_reg_sequencer_pkg;
2
3 import uvm_pkg::*;
4 `include "uvm_macros.svh"
5
6 import shift_reg_seq_item_pkg::*;
7 class shift_reg_sequencer extends uvm_sequencer #(shift_reg_seq_item);
8     `uvm_component_utils(shift_reg_sequencer);
9
10     function new (string name = "shift_reg_sequencer", uvm_component parent = null);
11         super.new(name, parent);
12     endfunction
13 endclass
14
15 endpackage
```

31. Src file:

```

1  shift_reg_config_pkg.sv
2  alsu_config_pkg.sv
3  shared_pkg_alasu.sv
4  shift_reg_if.sv
5  alsu_if.sv
6  shift_reg.sv
7  ALSU.sv
8  shift_reg_seq_item_pkg.sv
9  shift_reg_sequencer_pkg.sv
10 shift_reg_driver_pkg.sv
11 shift_reg_monitor_pkg.sv
12 shift_reg_agent_pkg.sv
13 shift_reg_coverage_pkg.sv
14 shift_reg_scoreboard_pkg.sv
15 shift_reg_env_pkg.sv
16 shift_reg_main_seq_pkg.sv
17 golden_design.sv
18 alsu_seq_item_pkg.sv
19 alsu_sequencer_pkg.sv
20 alsu_driver_pkg.sv
21 alsu_monitor_pkg.sv
22 alsu_agent_pkg.sv
23 alsu_coverage_pkg.sv
24 alsu_scoreboard_pkg.sv
25 alsu_env_pkg.sv
26 alsu_reset_seq_pkg.sv
27 alsu_main_seq_pkg.sv
28 alsu_test_pkg.svq
29 alsu_sva.sv
30 alsu_top.sv

```

32. Coverage text of alsu:

Directive Coverage:					
Directives	19	19	0	100.00%	
DIRECTIVE COVERAGE:					
Name	Design Unit	Design UnitType	Lang	File(Line)	Hits Status
/alsu_top/dut/sva_inst/c1	alsu_sva	Verilog	SVA	alsu_sva.sv(6)	92 Covered
/alsu_top/dut/sva_inst/c2	alsu_sva	Verilog	SVA	alsu_sva.sv(11)	2004 Covered
/alsu_top/dut/sva_inst/c3	alsu_sva	Verilog	SVA	alsu_sva.sv(16)	1822 Covered
/alsu_top/dut/sva_inst/c4	alsu_sva	Verilog	SVA	alsu_sva.sv(23)	24180 Covered
/alsu_top/dut/sva_inst/c5	alsu_sva	Verilog	SVA	alsu_sva.sv(28)	1608 Covered
/alsu_top/dut/sva_inst/c6	alsu_sva	Verilog	SVA	alsu_sva.sv(33)	3364 Covered
/alsu_top/dut/sva_inst/c7	alsu_sva	Verilog	SVA	alsu_sva.sv(38)	3244 Covered
/alsu_top/dut/sva_inst/c8	alsu_sva	Verilog	SVA	alsu_sva.sv(43)	2050 Covered
/alsu_top/dut/sva_inst/c9	alsu_sva	Verilog	SVA	alsu_sva.sv(48)	580 Covered
/alsu_top/dut/sva_inst/c10	alsu_sva	Verilog	SVA	alsu_sva.sv(53)	1200 Covered
/alsu_top/dut/sva_inst/c11	alsu_sva	Verilog	SVA	alsu_sva.sv(58)	988 Covered
/alsu_top/dut/sva_inst/c12	alsu_sva	Verilog	SVA	alsu_sva.sv(63)	2004 Covered
/alsu_top/dut/sva_inst/c13	alsu_sva	Verilog	SVA	alsu_sva.sv(68)	2012 Covered
/alsu_top/dut/sva_inst/c14	alsu_sva	Verilog	SVA	alsu_sva.sv(73)	2014 Covered
/alsu_top/dut/sva_inst/c15	alsu_sva	Verilog	SVA	alsu_sva.sv(78)	958 Covered
/alsu_top/dut/sva_inst/c16	alsu_sva	Verilog	SVA	alsu_sva.sv(84)	1054 Covered
/alsu_top/dut/sva_inst/c17	alsu_sva	Verilog	SVA	alsu_sva.sv(89)	954 Covered
/alsu_top/dut/sva_inst/c18	alsu_sva	Verilog	SVA	alsu_sva.sv(94)	1054 Covered
/alsu_top/dut/sva_inst/c19	alsu_sva	Verilog	SVA	alsu_sva.sv(100)	26910 Covered

```

828
829 Toggle Coverage:
830   Enabled Coverage      Bins      Hits      Misses Coverage
831   -----
832   Toggles               38       38       0    100.00%
833
834 =====Toggle Details=====
835
836 Toggle Coverage for instance /alsu_top/dut --
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859

```

Node	1H->0L	0L->1H	"Coverage"
A_reg[2-0]	1	1	100.00
B_reg[2-0]	1	1	100.00
bypass_A_reg	1	1	100.00
bypass_B_reg	1	1	100.00
cin_reg[0]	1	1	100.00
direction_reg	1	1	100.00
invalid	1	1	100.00
invalid_opcode	1	1	100.00
invalid_red_op	1	1	100.00
opcode_reg[2-0]	1	1	100.00
red_op_A_reg	1	1	100.00
red_op_B_reg	1	1	100.00
serial_in_reg	1	1	100.00

```

Total Node Count      =      19
Toggled Node Count    =      19
Untoggled Node Count  =       0
Toggle Coverage       =    100.00% (38 of 38 bins)

```

```

53
54 Assertion Coverage:
55   Assertions              20      20      0    100.00%
56   -----
57   Name                    File(Line)                    Failure    Pass
58   Count                    Count
59   -----
60 /alsu_top/dut/sva_inst/a1
61   alsu_sva.sv(4)              0          1
62 /alsu_top/dut/sva_inst/a2
63   alsu_sva.sv(9)              0          1
64 /alsu_top/dut/sva_inst/a3
65   alsu_sva.sv(14)             0          1
66 /alsu_top/dut/sva_inst/a4
67   alsu_sva.sv(20)             0          1
68 /alsu_top/dut/sva_inst/a5
69   alsu_sva.sv(26)             0          1
70 /alsu_top/dut/sva_inst/a6
71   alsu_sva.sv(31)             0          1
72 /alsu_top/dut/sva_inst/a7
73   alsu_sva.sv(36)             0          1
74 /alsu_top/dut/sva_inst/a8
75   alsu_sva.sv(41)             0          1
76 /alsu_top/dut/sva_inst/a9
77   alsu_sva.sv(46)             0          1
78 /alsu_top/dut/sva_inst/a10
79   alsu_sva.sv(51)             0          1
80 /alsu_top/dut/sva_inst/a11
81   alsu_sva.sv(56)             0          1
82 /alsu_top/dut/sva_inst/a12
83   alsu_sva.sv(61)             0          1
84 /alsu_top/dut/sva_inst/a13
85   alsu_sva.sv(66)             0          1
86 /alsu_top/dut/sva_inst/a14
87   alsu_sva.sv(71)             0          1
88 /alsu_top/dut/sva_inst/a15
89   alsu_sva.sv(76)             0          1
90 /alsu_top/dut/sva_inst/a16
91   alsu_sva.sv(82)             0          1
92 /alsu_top/dut/sva_inst/a17
93   alsu_sva.sv(87)             0          1
94 /alsu_top/dut/sva_inst/a18
95   alsu_sva.sv(92)             0          1
96 /alsu_top/dut/sva_inst/a19
97   alsu_sva.sv(97)             0          1
98 /alsu_top/dut/sva_inst/o_1
99   alsu_sva.sv(105)            0          1

```



```

370 === Design Unit: work.ALSU
371 =====
372 Branch Coverage:
373   Enabled Coverage      Bins    Hits    Misses  Coverage
374   -----
375   Branches              32      31      1      96.87%
376
377   =====Branch Details=====
378
379 Branch Coverage for instance /alsu_top/dut
380
381   Line      Item      Count      Source
382   ----      -
383   File ALSU.sv
384   -----IF Branch-----
385   15              51959  Count coming in to IF
386   15              407    if(a_if.rst) begin
387
388   26              51552  end else begin
389
390 Branch totals: 2 hits of 2 branches = 100.00%
391
392   -----IF Branch-----
393   42              52204  Count coming in to IF
394   42              612    if(a_if.rst) begin
395
396   44              51592  end else begin
397
398 Branch totals: 2 hits of 2 branches = 100.00%
399
400   -----IF Branch-----
401   45              51592  Count coming in to IF
402   45              27057  if (invalid)
403
404   47              24535  else
405
406 Branch totals: 2 hits of 2 branches = 100.00%
407
408   -----IF Branch-----
409   54              52204  Count coming in to IF
410   54              612    if(a_if.rst) begin
411
412   57              51592  else begin
413
414 Branch totals: 2 hits of 2 branches = 100.00%
415
416   -----IF Branch-----
417   58              51592  Count coming in to IF
418   58              92     if (bypass_A_reg && bypass_B_reg)
419
420   60              2013  else if (bypass_A_reg)
421
422   62              1830  else if (bypass_B_reg)
423
424 Branch totals: 5 hits of 5 branches = 100.00%
425
426   -----CASE Branch-----
427   67              23346  Count coming in to CASE
428   68              10510  3'h0: begin
429
430   78              4788   3'h1: begin
431
432   88              2013   3'h2: begin
433
434   94              2014   3'h3: a_if.out <= A_reg * B_reg;
435
436   95              2012   3'h4: begin
437
438   101             2009   3'h5: begin
439
440   ****0*** All False Count
441
442 Branch totals: 6 hits of 7 branches = 85.71%
443
444   -----IF Branch-----
445   69              10510  Count coming in to IF
446   69              1615  if (red_op_A_reg && red_op_B_reg)
447
448   71              3264   else if (red_op_A_reg)
449
450   73              3379   else if (red_op_B_reg)
451
452   75              2252   else
453
454 Branch totals: 4 hits of 4 branches = 100.00%
455
456   -----IF Branch-----
457   79              4788  Count coming in to IF
458   79              580   if (red_op_A_reg && red_op_B_reg)
459
460   81              998   else if (red_op_A_reg)

```

Branch coverage and total coverage are not 100% because there is no default case. Instead, a **false count** statement exists. When the opcode takes an default value, the false count becomes one. However, the default opcode values 6 and 7 are already handled separately as invalid, outside the

case statement. Therefore, when the opcode is 6 or 7, the default case is never reached, and the false count will never be triggered.

```

1
2
3 Statement Coverage:
4 Enabled Coverage      Bins      Hits      Misses      Coverage
5 -----
6 Statements            48        48         0      100.00%
7
8 -----Statement Details-----
9
10 Statement Coverage for instance /alsu_top/dut --
11
12 Line      Item      Count      Source
13 ----      -
14 File ALSU.sv
15 3
16                                module ALSU(alsu_if.DUT a_if);
17
18 4                                reg red_op_A_reg, red_op_B_reg, bypass_A_reg, bypass_B_reg, direction_reg, serial_in_reg;
19
20 5                                reg [1:0] cin_reg;
21
22 6                                reg [2:0] opcode_reg;
23
24 7                                reg signed [2:0] A_reg, B_reg;
25
26 8                                wire invalid_red_op, invalid_opcode, invalid;
27
28 9                                //Invalid handling
29
30 10           1                23924    assign invalid_red_op = (red_op_A_reg || red_op_B_reg) & (opcode_reg[1] || opcode_reg[2]);

```

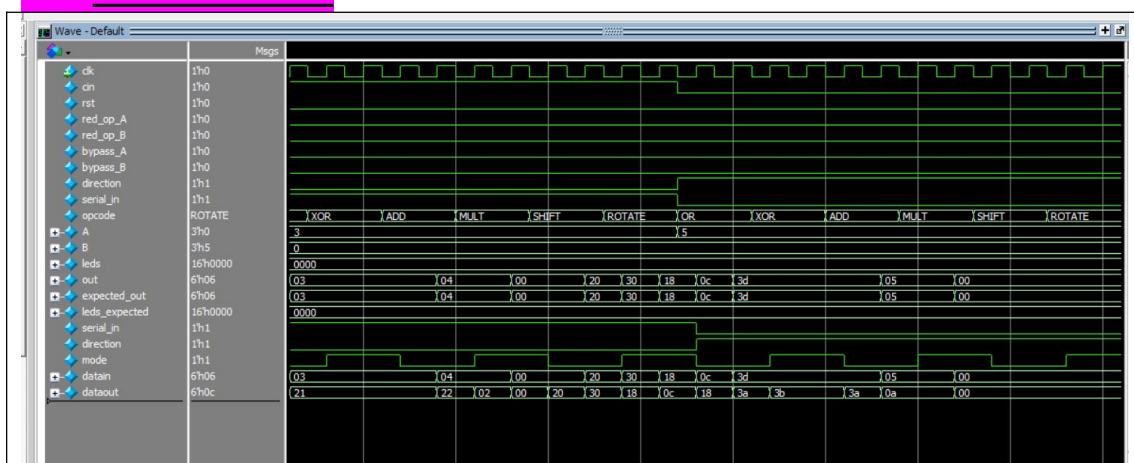
```

1663
1664 Covergroup Coverage:
1665 Covergroups            1      na      na      100.00%
1666 Coverpoints/Crosses    18      na      na      na
1667 Covergroup Bins        46      46      0      100.00%
1668
1669 -----
1670 Covergroup              Metric      Goal      Bins      Status
1671 -----
1672 TYPE /alsu_coverage_pkg/alsu_coverage/alsu_cg      100.00%      100      -      Covered
1673 covered/total bins:      46      46      -
1674 missing/total bins:      0      46      -
1675 % Hit:      100.00%      100      -
1676 Coverpoint A_cp1      100.00%      100      -      Covered
1677 covered/total bins:      3      3      -
1678 missing/total bins:      0      3      -
1679 % Hit:      100.00%      100      -
1680 Coverpoint A_cp2      100.00%      100      -      Covered
1681 covered/total bins:      2      2      -
1682 missing/total bins:      0      2      -
1683 % Hit:      100.00%      100      -
1684 Coverpoint B_cp1      100.00%      100      -      Covered
1685 covered/total bins:      3      3      -
1686 missing/total bins:      0      3      -
1687 % Hit:      100.00%      100      -
1688 Coverpoint B_cp2      100.00%      100      -      Covered
1689 covered/total bins:      2      2      -
1690 missing/total bins:      0      2      -
1691 % Hit:      100.00%      100      -
1692 Coverpoint ALU_cp      100.00%      100      -      Covered
1693 covered/total bins:      8      8      -
1694 missing/total bins:      0      8      -
1695 % Hit:      100.00%      100      -
1696 Coverpoint c_cp      100.00%      100      -      Covered
1697 covered/total bins:      2      2      -
1698 missing/total bins:      0      2      -
1699 % Hit:      100.00%      100      -
1700 Coverpoint direction_cp      100.00%      100      -      Covered
1701 covered/total bins:      2      2      -
1702 missing/total bins:      0      2      -
1703 % Hit:      100.00%      100      -
1704 Coverpoint serial_cp      100.00%      100      -      Covered
1705 covered/total bins:      2      2      -
1706 missing/total bins:      0      2      -
1707 % Hit:      100.00%      100      -
1708 Coverpoint reda_cp      100.00%      100      -      Covered
1709 covered/total bins:      2      2      -
1710 missing/total bins:      0      2      -
1711 % Hit:      100.00%      100      -
1712 Coverpoint redb_cp      100.00%      100      -      Covered
1713 covered/total bins:      2      2      -
1714 missing/total bins:      0      2      -
1715 % Hit:      100.00%      100      -
1716 Cross c1_cs      100.00%      100      -      Covered
1717 covered/total bins:      3      3      -

```

1826	% Hit:	100.00%	100	-	
1827	Auto, Default and User Defined Bins:				
1828	bin add_mult_b1	2432	1	-	Covered
1829	bin add_mult_b2	1915	1	-	Covered
1830	bin add_mult_b3	1418	1	-	Covered
1831	Cross c2_cs	100.00%	100	-	Covered
1832	covered/total bins:	3	3	-	
1833	missing/total bins:	0	3	-	
1834	% Hit:	100.00%	100	-	
1835	Auto, Default and User Defined Bins:				
1836	bin add_mult_a1	2445	1	-	Covered
1837	bin add_mult_a2	1998	1	-	Covered
1838	bin add_mult_a3	1402	1	-	Covered
1839	Cross c3_cs	100.00%	100	-	Covered
1840	covered/total bins:	1	1	-	
1841	missing/total bins:	0	1	-	
1842	% Hit:	100.00%	100	-	
1843	Auto, Default and User Defined Bins:				
1844	bin add	3765	1	-	Covered
1845	Cross c4_cs	100.00%	100	-	Covered
1846	covered/total bins:	2	2	-	
1847	missing/total bins:	0	2	-	
1848	% Hit:	100.00%	100	-	
1849	Auto, Default and User Defined Bins:				
1850	bin sh	3713	1	-	Covered
1851	bin r'	3708	1	-	Covered
1852	Cross c5_cs	100.00%	100	-	Covered
1853	covered/total bins:	1	1	-	
1854	missing/total bins:	0	1	-	
1855	% Hit:	100.00%	100	-	
1856	Auto, Default and User Defined Bins:				
1857	bin sh	3713	1	-	Covered
1858	Cross c6_cs	100.00%	100	-	Covered
1859	covered/total bins:	1	1	-	
1860	missing/total bins:	0	1	-	
1861	% Hit:	100.00%	100	-	
1862	Auto, Default and User Defined Bins:				
1863	bin reda	3247	1	-	Covered
1864	Cross c7_cs	100.00%	100	-	Covered
1865	covered/total bins:	1	1	-	
1866	missing/total bins:	0	1	-	
1867	% Hit:	100.00%	100	-	
1868	Auto, Default and User Defined Bins:				
1869	bin redb	3404	1	-	Covered
1870	Cross c8_cs	100.00%	100	-	Covered
1871	covered/total bins:	6	6	-	
1872	missing/total bins:	0	6	-	
1873	% Hit:	100.00%	100	-	
1874	Auto, Default and User Defined Bins:				
1875	bin a1	217	1	-	Covered
1876	bin a2	170	1	-	Covered
1877	bin b1	217	1	-	Covered
1878	bin b2	210	1	-	Covered
1879	bin ab1	4976	1	-	Covered
1880	bin ab2	5209	1	-	Covered

33. Wave form :



34. Output :

```

# UVM_INFO alsu_scoreboard_pkg.sv(35) @ 104004: uvm_test_top.a_env.score_b [run_phase] expected_out value : 000110 , leds_expected : 0000000000000000
# UVM_INFO alsu_test_pkg.sv(61) @ 104004: uvm_test_top [run_phase] Stimulus Generation Ended
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 104004: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO alsu_scoreboard_pkg.sv(54) @ 104004: uvm_test_top.a_env.score_b [report_phase] Total successful transactions: 26001
# UVM_INFO alsu_scoreboard_pkg.sv(55) @ 104004: uvm_test_top.a_env.score_b [report_phase] Total failed transactions: 0
# UVM_INFO shift_reg_scoreboard_pkg.sv(68) @ 104004: uvm_test_top.s_env.sb [report_phase] Total successful transactions: 52002
# UVM_INFO shift_reg_scoreboard_pkg.sv(69) @ 104004: uvm_test_top.s_env.sb [report_phase] Total failed transactions: 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :52014
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [report_phase] 4
# [run_phase] 52006
#
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 104004 ns Iteration: 61 Instance: /alsu_top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430

```

35. Do file:

```

1 vlib work
2 vlog -f src_files.list.txt +cover -covercells
3 vsim -voptargs=+acc work.alsu_top -classdebug -uvmcontrol=all -cover
4 add wave /alsu_top/a_if/*
5 add wave /alsu_top/shiftregif/*
6 coverage save alsu_top.ucdb -onexit
7 run -all
8 coverage exclude -du ALSU -togglenode {cin_reg[1]}

```

36. Shift reg coverage text:

```

=== Instance: /alsu_top/DUT
=== Design Unit: work.shift_reg
=====
Branch Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Branches              5      5      0    100.00%

=====Branch Details=====
Branch Coverage for instance /alsu_top/DUT
  Line    Item              Count    Source
  ----    -
  File shift_reg.sv
  -----IF Branch-----
  13              50205    Count coming in to IF
  13              21373    if (shiftregif.mode) // rotate
  19              14323    if (shiftregif.direction) // left
  21              14509    else

Branch totals: 3 hits of 3 branches = 100.00%

  -----IF Branch-----
  14              21373    Count coming in to IF
  14              10694    if (shiftregif.direction) // left
  16              10679    else

Branch totals: 2 hits of 2 branches = 100.00%

Statement Coverage:
  Enabled Coverage      Bins    Hits    Misses  Coverage
  -----
  Statements             5      5      0    100.00%

```


3852	=====						
3853							
3854	Covergroup Coverage:						
3855	Covergroups	1	na	na	98.43%		
3856	Coverpoints/Crosses	4	na	na	na		
3857	Covergroup Bins	70	66	4	94.28%		
3858	-----						
3859	Covergroup			Metric	Goal	Bins	Status
3860							
3861	-----						
3862	TYPE /shift_reg_coverage_pkg/shift_reg_coverage/cvr_grp						
3863				98.43%	100	-	Uncovered
3864	covered/total bins:			66	70	-	
3865	missing/total bins:			4	70	-	
3866	% Hit:			94.28%	100	-	
3867	Coverpoint datain_cvp			93.75%	100	-	Uncovered
3868	covered/total bins:			60	64	-	
3869	missing/total bins:			4	64	-	
3870	% Hit:			93.75%	100	-	
3871	bin auto[0]			44743	1	-	Covered
3872	bin auto[1]			13739	1	-	Covered
3873	bin auto[2]			1119	1	-	Covered
3874	bin auto[3]			1915	1	-	Covered
3875	bin auto[4]			306	1	-	Covered
3876	bin auto[5]			202	1	-	Covered
3877	bin auto[6]			443	1	-	Covered
3878	bin auto[7]			308	1	-	Covered
3879	bin auto[8]			334	1	-	Covered
3880	bin auto[9]			263	1	-	Covered
3881	bin auto[10]			152	1	-	Covered
3882	bin auto[11]			150	1	-	Covered
3883	bin auto[12]			356	1	-	Covered
3884	bin auto[13]			149	1	-	Covered
3885	bin auto[14]			86	1	-	Covered
3886	bin auto[15]			114	1	-	Covered
3887	bin auto[16]			152	1	-	Covered
3888	bin auto[17]			19	1	-	Covered
3889	bin auto[18]			10	1	-	Covered
3890	bin auto[19]			72	1	-	Covered
3891	bin auto[20]			0	1	-	ZERO
3892	bin auto[21]			0	1	-	ZERO
3893	bin auto[22]			16	1	-	Covered
3894	bin auto[23]			13	1	-	Covered
3895	bin auto[24]			128	1	-	Covered
3896	bin auto[25]			19	1	-	Covered
3897	bin auto[26]			19	1	-	Covered
3898	bin auto[27]			26	1	-	Covered
3899	bin auto[28]			51	1	-	Covered
3900	bin auto[29]			18	1	-	Covered
3901	bin auto[30]			44	1	-	Covered
3902	bin auto[31]			88	1	-	Covered
3903	bin auto[32]			194	1	-	Covered

There is some bins from the auto generated bins did not hit